

Notes on Parameters for Hyperparameter Optimization in Machine Learning

Luis Mendez

PARAMETERS IN ML MODELS

The objective of a typical learning algorithm is to find a function f that maps input data X to output labels Y . The function f is defined by a set of parameters θ , which are learned from the training data. For example, in a linear regression model, the parameters are the coefficients of the linear equation. In a neural network, the parameters are the weights and biases of the network. The learning algorithm adjusts these parameters to minimize a loss function, which measures the difference between the predicted output and the true output.

The coefficients of the linear function, are the parameters to find or optimise by the algorithm. The learning algorithm adjusts these parameters to minimize a loss function, which measures the difference between the predicted output and the true output.

Hyperparameters are parameters that are not directly learned from by the learning algorithm, hyperparameters are specified outside of the training procedure, they control the capacity of the model, i.e., how flexible the model is to fit the data, and they prevent over-fitting. Examples of hyperparameters include the learning rate, the number of hidden layers in a neural network, the number of trees in a random forest, and the regularization strength in a linear model. Hyperparameter optimization is the process of finding the best set of hyperparameters for a given model and dataset, which can significantly improve the performance of the model.

LINEAR REGRESSION HYPERPARAMETERS

In linear regression, the main hyperparameters include:

- **Regularization strength** (λ): This hyperparameter controls the amount of regularization applied to the model. Regularization helps prevent overfitting by adding a penalty term to the loss function. Common types of regularization include L1 (Lasso) and L2 (Ridge) regularization.
- **Learning rate** (α): This hyperparameter controls the step size at which the learning algorithm updates the parameters during training. A smaller learning rate may lead to slower convergence, while a larger learning rate may cause the algorithm to overshoot the optimal parameters.
- **Number of iterations** (n_{iter}): This hyperparameter specifies the number of times the learning algorithm will iterate over the training data. More iterations can lead to better convergence, but it may also increase training time.

Ridge penalizes the sum of the squares of the coefficients, while Lasso penalizes the sum of the absolute values of the

coefficients. The choice of regularization type and strength can significantly impact the performance of the linear regression model, especially when dealing with high-dimensional data or multicollinearity. Lasso penalizes the sum of the absolute values of the coefficients, which can lead to sparse solutions where some coefficients are exactly zero. This can be useful for feature selection, as it effectively removes less important features from the model. Ridge, on the other hand, penalizes the sum of the squares of the coefficients, which tends to shrink the coefficients towards zero but does not set them exactly to zero. This can be beneficial when dealing with multicollinearity, as it helps to stabilize the estimates of the coefficients.

We need to decide the regularization penalty and the strength of the regularization, which can be done through hyperparameter optimization techniques such as grid search or random search.

DECISION TREE HYPERPARAMETERS

In decision trees, the main hyperparameters include:

- **Max depth** (max_depth): This hyperparameter controls the maximum depth of the tree. A deeper tree can capture more complex patterns in the data but may also lead to overfitting.
- **Min samples split** ($min_samples_split$): This hyperparameter specifies the minimum number of samples required to split an internal node. A higher value can prevent the tree from growing too deep and overfitting.
- **Min samples leaf** ($min_samples_leaf$): This hyperparameter specifies the minimum number of samples required to be at a leaf node. A higher value can help to smooth the model and prevent overfitting.
- **Max features** ($max_features$): This hyperparameter controls the number of features to consider when looking for the best split. A smaller value can reduce overfitting by introducing randomness into the model.

The choice of these hyperparameters can significantly impact the performance of the decision tree model. For example, a very deep tree may fit the training data perfectly but perform poorly on unseen data due to overfitting. On the other hand, a shallow tree may not capture enough complexity in the data, leading to underfitting. Therefore, it is important to carefully tune these hyperparameters to achieve the best performance on the validation set.

In a decision tree, the bigger the depth the best the prediction, but we may have overfitting.

HYPERPARAMETERS OPTIMIZATION TECHNIQUES

There are several techniques for hyperparameter optimization, including:

- **Grid Search:** This technique involves defining a grid of hyperparameter values and exhaustively searching through all possible combinations to find the best set of hyperparameters based on a performance metric (e.g., accuracy, F1 score).
- **Random Search:** This technique involves randomly sampling hyperparameter values from a specified distribution. It can be more efficient than grid search, especially when the hyperparameter space is large.
- **Bayesian Optimization:** This technique uses a probabilistic model to predict the performance of different hyperparameter combinations and selects the next set of hyperparameters to evaluate based on this model. It can be more efficient than random search and grid search, especially for expensive-to-evaluate models.
- **Genetic Algorithms:** This technique uses evolutionary algorithms to optimize hyperparameters by simulating the process of natural selection. It can be effective for complex hyperparameter spaces.

The choice of hyperparameter optimization technique depends on the size of the hyperparameter space, the computational resources available, and the specific requirements of the problem at hand. Grid search can be effective for small hyperparameter spaces, while random search and Bayesian optimization can be more efficient for larger spaces. Genetic algorithms can be useful for complex hyperparameter spaces where traditional optimization techniques may struggle.

RANDOM FOREST HYPERPARAMETERS

- **Number of trees** (*n_estimators*): This hyperparameter controls the number of trees in the random forest. A larger number of trees can improve performance but also increases computational cost.
- **Max depth** (*max_depth*): Similar to decision trees, this hyperparameter controls the maximum depth of each tree in the random forest.
- **Min samples split** (*min_samples_split*): This hyperparameter specifies the minimum number of samples required to split an internal node in each tree.
- **Min samples leaf** (*min_samples_leaf*): This hyperparameter specifies the minimum number of samples required to be at a leaf node in each tree.
- **Max features** (*max_features*): This hyperparameter controls the number of features to consider when looking for the best split in each tree. A smaller value can reduce overfitting by introducing randomness into the model.

The choice of these hyperparameters can significantly impact the performance of the random forest model. For example, a larger number of trees can improve the model's performance by reducing variance, but it also increases computational cost. Similarly, tuning the max depth and min samples parameters can help to prevent overfitting and improve generalization to unseen data.